40 Days of LeetCode Challenge

40 days, 40 problems
DAY :5

LEETCODE PROBLEM NO. 728 : Self Dividing Number

Problem Link : https://leetcode.com/problems/self-dividing-numbers/description/
Easy · Digit manipulation · arithmetic operators · Mathematical Logic
STEP BY STEP SOLUTION:-

Step 1. Understand the question :-

We must return a list of all self-dividing numbers within that range, including both boundaries.

A number is called self-dividing if:

- It is divisible by every digit it contains.
- It does not contain the digit 0, because division by zero is undefined.

For example, 128 is self-dividing because:

- (128 mod 1=0)
- (128 mod 2=0)
- (128 mod 8=0)

All three digits divide the number exactly, so 128 qualifies.

examples :- The number 124 qualify because the digits 1, 2, 4 divides 124 evenly. Similarly 126 is also self-dividing because its digits are 1, 2, and 6, all of which divide it evenly and 123 is also self-dividing because its digits are 1, 2, and 3, all of which divide it evenly.

But the number 127: the digit 7 does not divide 127 evenly. Therefore, 127 is not self-dividing.

Since (127mod 7=1).

Step 2. Translating it into simpler language with examples :-

Example 1:
128 is a self-dividing number because 128 % 1 == 0, 128 % 2 == 0, and 128 % 8 == 0.

n = 128
Qualifies ✓

- 128 % 1 = 0
- 128 % 2 = 0
- 128 % 8 = 0

Every digit passes!

Example 2:
127 and 105

n = 127

Doesn't qualify ✗

- 127 % 1 = 0
- 127 % 2 = 1
- 127 % 7 = 1

The digit 2 and the digit 7 fail the divisibility test.

n = 105

Doesn't qualify ✗

It contains 0, so it cannot be self dividing. We must never attempt division by zero.

Step 3. Identify the pattern

Today's problem combines two familiar concepts:

- Range traversal: check every number from left to right, inclusive.
- Digit manipulation: extract each digit and check whether it divides the original number.

Operator	Purpose	Example
% 10	Extract the last digit	128 % 10 = 8
/ 10	Remove the last digit using integer division	128 / 10 = 12
% digit	Check divisibility	

NOTE :-
Keep the original number unchanged for divisibility checks. Use a separate temporary variable to extract and remove digits.

If we overwrite the original number while extracting digits, we might end up checking the wrong value

A. Modulo operator %

The expression n % 10 returns the last digit of a positive integer.

For example:

- 121 % 10 = 1
- 12 % 10 = 2
- 1 % 10 = 1

B. Integer division /

The expression n / 10 removes the last digit when both operands are integers and n is non-negative.

For example:

- 121 / 10 = 12
- 12 / 10 = 1
- 1 / 10 = 0

Together, these operations allow us to process the digits from right to left without converting the number into a stri ng.

Step 4. Build the logic

Here's the approach we'll follow:

Step A: Create an answer list

We need to return all valid numbers, not just one. In Java, we'll use an ArrayList<Integer> to store them.

Step B: Check every number in the range

Start from left and continue until right, including right.

For each number, assume it is self-dividing until we discover a reason to reject it.

Step C: Extract its digits

Create a temporary variable temp = num.

Repeatedly:

1. Extract the last digit: digit = temp % 10.
2. If digit == 0, reject the number.
3. Otherwise, check num % digit. If the remainder isn't zero, reject it.
4. Remove the last digit: temp /= 10.

Step D: Store only valid numbers

If every digit passes the checks, add the original num to the answer list.

Algorithm

1. Start.
2. Create an empty list called result.
3. Iterate through each number num from left to right.
4. Initialize temp = num and isSelfDividing = true.
5. While temp > 0:
    1. Extract the last digit using temp % 10.
    2. If the digit is zero, or num % digit != 0, set isSelfDividing = false and
    3. stop checking this number.
    4. Otherwise, remove the last digit using temp / 10.

6. If isSelfDividing is true, add num to result.
7. Return result.
8. Stop.

Pseudocode

```
ALGORITHM SelfDividingNumbers(left, right)

BEGIN
  result ← empty list

  FOR num ← left TO right DO
    temp ← num
    isSelfDividing ← true

    WHILE temp > 0 DO
      digit ← temp MOD 10

      IF digit = 0 OR num MOD digit ≠ 0 THEN
        isSelfDividing ← false
        BREAK
      END IF

      temp ← temp DIV 10
    END WHILE

    IF isSelfDividing = true THEN
      ADD num TO result
    END IF
  END FOR

  RETURN result
END
```

Step 5. Code in Java

This is the standard arithmetic solution for LeetCode 728.

```
import java.util.ArrayList;

import java.util.List;

class Solution {

  public List<Integer> selfDividingNumbers(int left, int right) {

    List<Integer> result = new ArrayList<>();

    for (int num = left; num <= right; num++) {

      int temp = num;
```

```
        boolean isSelfDividing = true;
        while (temp > 0) {
            int digit = temp % 10;


            if (digit == 0 || num % digit != 0) {
                isSelfDividing = false;
                break;
            }
            temp /= 10;
        }
        if (isSelfDividing) {
            result.add(num);
        }
    } return result;
  }
}
```

NOTE:-

Why do we need List<Integer>?

The method returns multiple numbers, so a list is suitable. List<Integer> is the interface,

and ArrayList<Integer> is the implementation we use to store the results dynamically.

Why use boolean isSelfDividing = true?

We begin by assuming the number is valid. If even one digit fails, we change the value to false.

Why use break?

As soon as a digit is zero or fails the divisibility check, there is no need to inspect the remaining digits of that number. break exits the inner loop immediately.

How to submit

1. Open LeetCode Problem 728. Self Dividing Numbers
2. Select Java as the programming language.

3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Explanation of the code :-

1. Import the required classes

import java.util.ArrayList;
import java.util.List;

ArrayList stores the qualifying numbers, while List provides the interface used as the method's return type.

2. Declare the method

public List<Integer> selfDividingNumbers(int left, int right)

The method accepts the range boundaries and returns a list of integers.

3. Initialize the result list

List<Integer> result = new ArrayList<>();

This list starts empty. Every self-dividing number we find will be added to it.

4. Traverse the range

for (int num = left; num <= right; num++)

This checks every number from left to right, including both boundaries.

5. Preserve the original number

int temp = num;
boolean isSelfDividing = true;

temp is used to extract digits without changing num. The Boolean flag tracks whether all digits pass the test.

6. Extract each digit

while (temp > 0) {
int digit = temp % 10;

The loop processes the number one digit at a time, starting from the right.

7. Check for invalid digits

if (digit == 0 || num % digit != 0)

This condition rejects the number if a digit is zero or if the original number is not divisible by that digit.

The zero check comes first because division or remainder by zero is undefined.

8. Reject the number and stop checking it

```
isSelfDividing = false;
break;
```

The number has failed at least one test, so we mark it invalid and exit the digit-processing loop.

9. Remove the processed digit

```
temp /= 10;
```

Integer division removes the final digit. For example, 128 / 10 becomes 12.

10. Add valid numbers to the list

```
if (isSelfDividing) {
   result.add(num);
}
```

Only numbers whose digits all pass the checks are added.

11. Return the answer

```
return result;
```

The method returns the complete list of self-dividing numbers.

Dry run: left = 1, right = 22

Let's trace three representative candidates.

Candidate: 12 :- Accepted

Step	Operation	Result
1	12 % 10	2
2	12 % 2	0
3	12 / 10	1
4	12 % 1	0

Both digits divide 12 exactly, so 12 is added to the result.

Candidate: 15 :- Accepted

Step	Operation	Result
1	15 % 10	5
2	15 % 5	0
3	15 / 10	1
4	15 % 1	0

Therefore, 15 is added to the result.

Candidate: 17 :- Rejected

Step	Operation	Result

1	17 % 10	7
2	17 % 7	3
3	17 / 10	1
4	17 % 1	0

Since the remainder isn't zero, the number fails immediately.

We set isSelfDividing = false and execute break.

After checking every number from 1 to 22, the output is:

[1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 22]

6. Analyze complexity and optimization

Time complexity: (O(d x n))

Let:

- (n = right-left+1), the number of integers in the range.
- (d) be the maximum number of digits in any number in the range.

We may examine up to (d) digits for every number, so the worst-case time complexity is (O(n X d)).

Since (d) is proportional to the number of digits

It can also be expressed as (O(n log right)) for positive bounds.

Auxiliary space complexity: (O(1))

Apart from the returned list, we use only a fixed number of variables, such as num, temp, digit, and isSelfDividing.

The output list is not counted as auxiliary space. If we include the returned list, the total space is (O(k)), where (k) is the number of self-dividing numbers found.

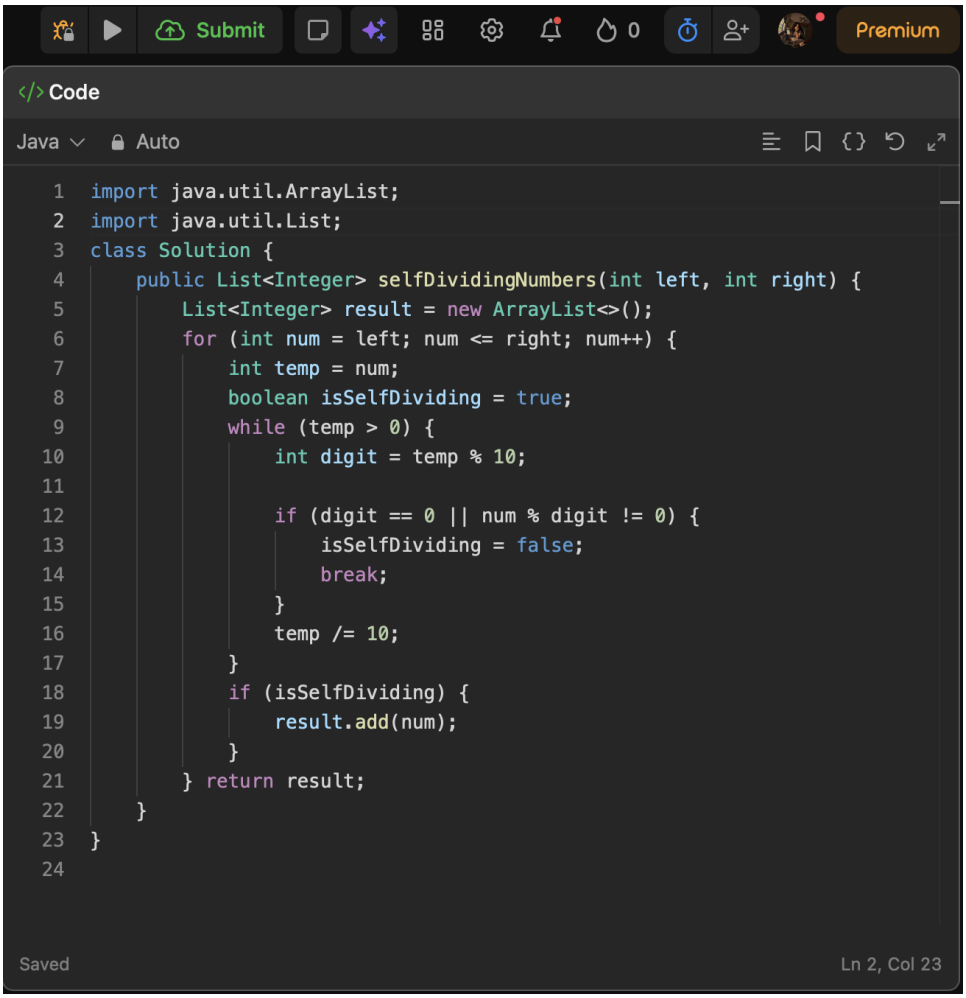
NOTE :-

Can we optimize it?

Our solution is already efficient for this problem. A few good practices help:

- Stop examining a candidate as soon as one digit fails, using break.
- Avoid converting numbers to strings because modulo and integer division are sufficient.
- Preserve the original number for divisibility checks.
- Don't add complicated mathematical tricks when the straightforward digit-checking approach is clear and efficient

SCREENSHOT:-


Submit
0
Premium
Code
Java
Auto
1 import java.util.ArrayList;
2 import java.util.List;
3 class Solution {
4 public List<Integer> selfDividingNumbers(int left, int right) {
5 List<Integer> result = new ArrayList<>();
6 for (int num = left; num <= right; num++) {
7 int temp = num;
8 boolean isSelfDividing = true;
9 while (temp > 0) {
10 int digit = temp % 10;
11
12 if (digit == 0 || num % digit != 0) {
13 isSelfDividing = false;
14 break;
15 }
16 temp /= 10;
17 }
18 if (isSelfDividing) {
19 result.add(num);
20 }
21 } return result;
22 }
23 }
24
Saved
Ln 2, Col 23